

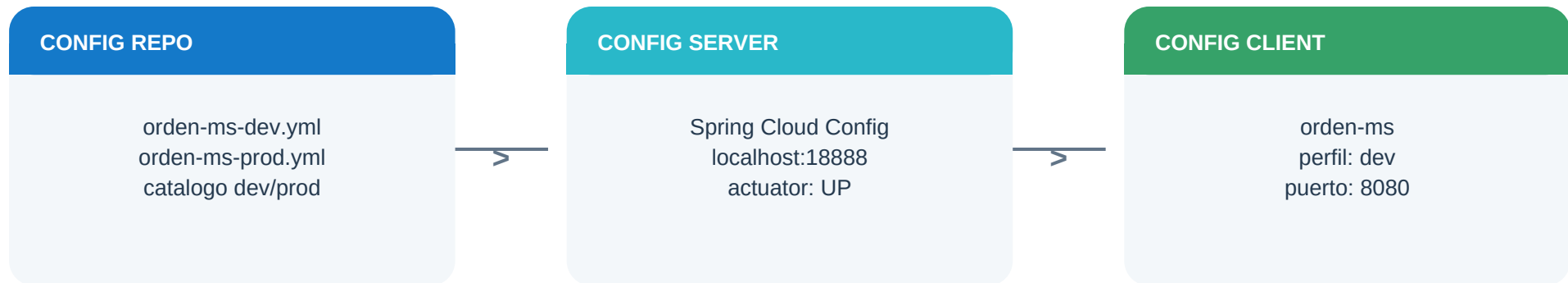
Gestion centralizada de configuracion y ambientes

Config Server, perfiles DEV/PROD y Config Client aplicados a los microservicios Pagatu.

Estudiante	Aldo Calla Ticona
Equipo	Sin grupo - trabajo individual
Rol / aporte	Implementacion y verificacion integral
Repositorio	github.com/Aldo-Ct/Pagalotu
Fecha	27 de agosto de 2026

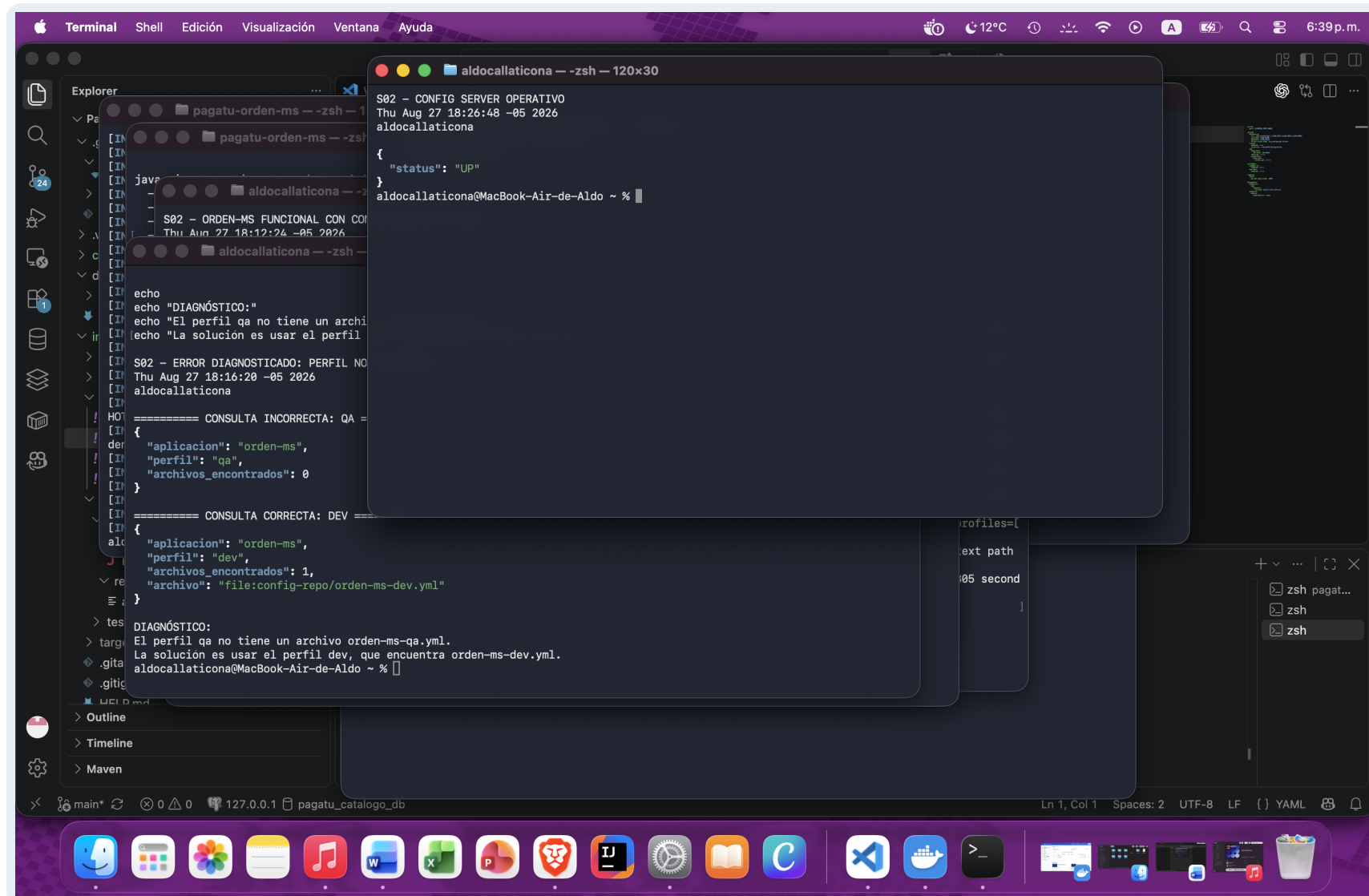
Objetivo, alcance y arquitectura verificada

Resultado: se centralizaron las propiedades por ambiente y el microservicio de ordenes consume su configuracion desde Config Server. Las pruebas demuestran salud, carga de perfiles, acceso a PostgreSQL y funcionamiento del endpoint REST.



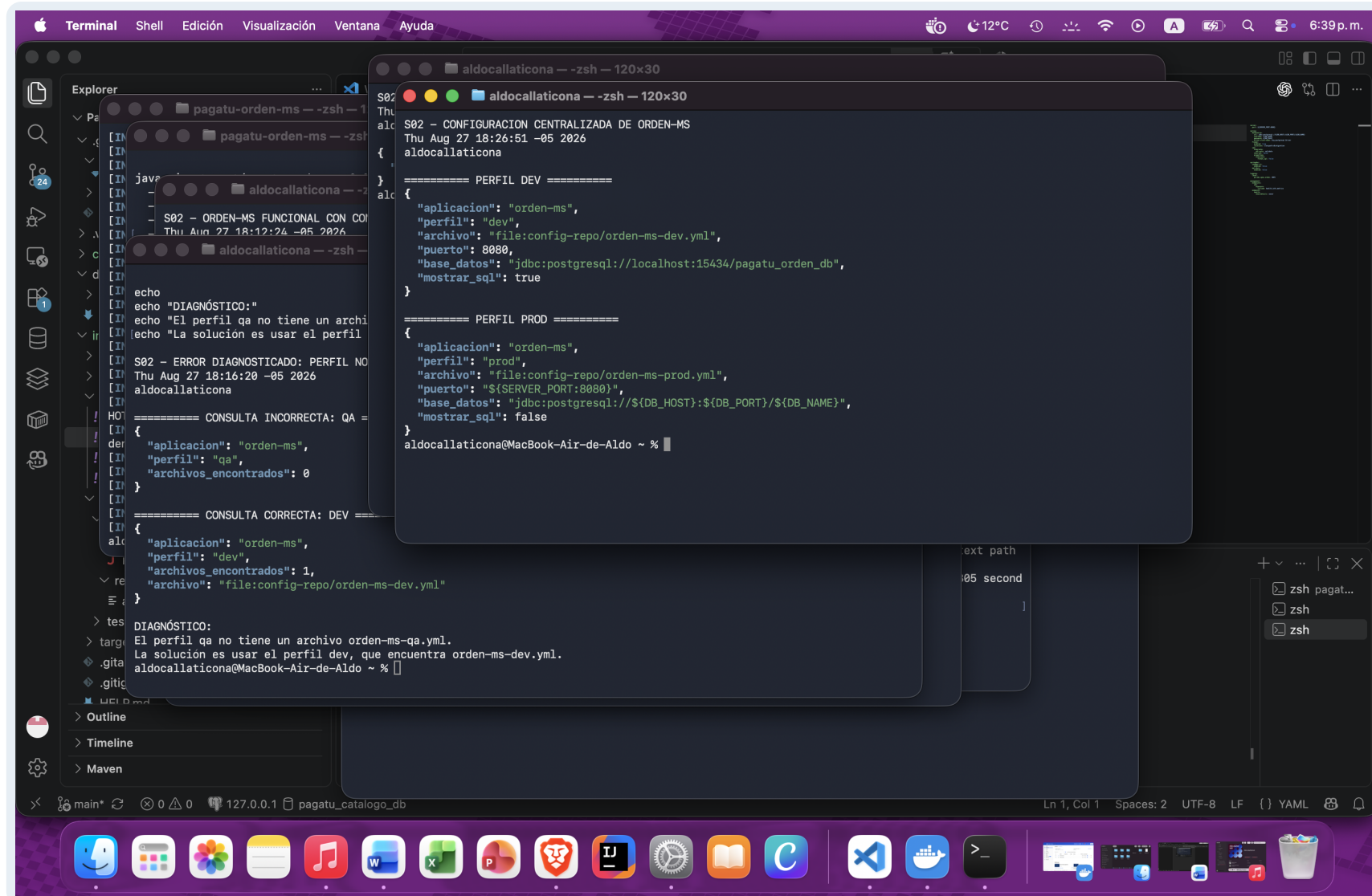
Criterio	Resultado	Evidencia
Config Server disponible	Cumplido	Health UP en :18888
Perfiles orden-ms DEV/PROD	Cumplido	PropertySources diferentes
orden-ms como Config Client	Cumplido	Carga remota y arranque en 8080
Funcionamiento con datos	Cumplido	DB UP y GET /api/v1/ordenes
Comparacion catalogo DEV/PROD	Cumplido	Valores reales externalizados

Config Server operativo



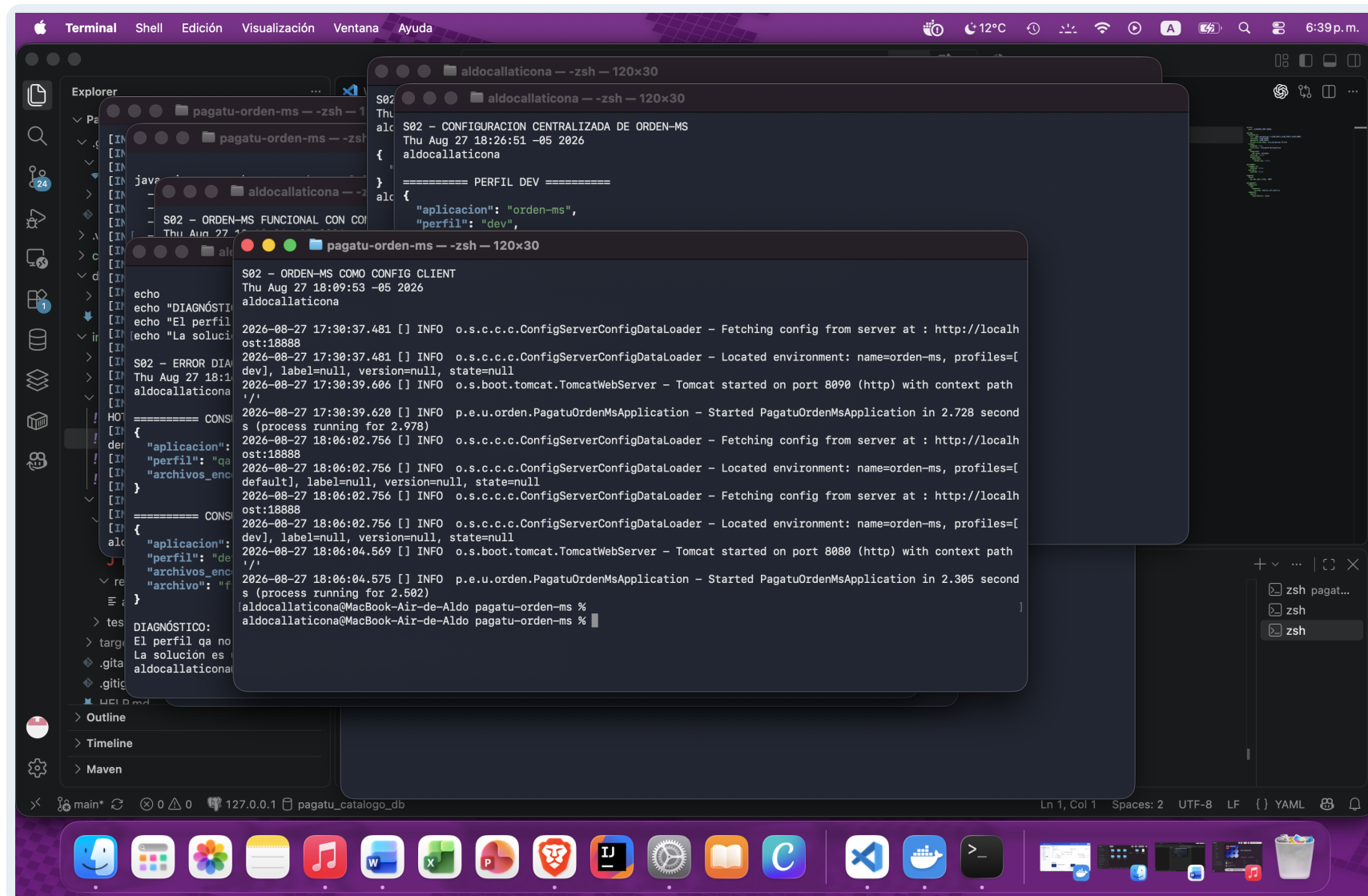
Evidencia: respuesta completa de /actuador/health, con fecha, hora y usuario visibles. **Hallazgo:** el servidor central responde UP en el puerto 18888.

Perfiles externos de orden-ms



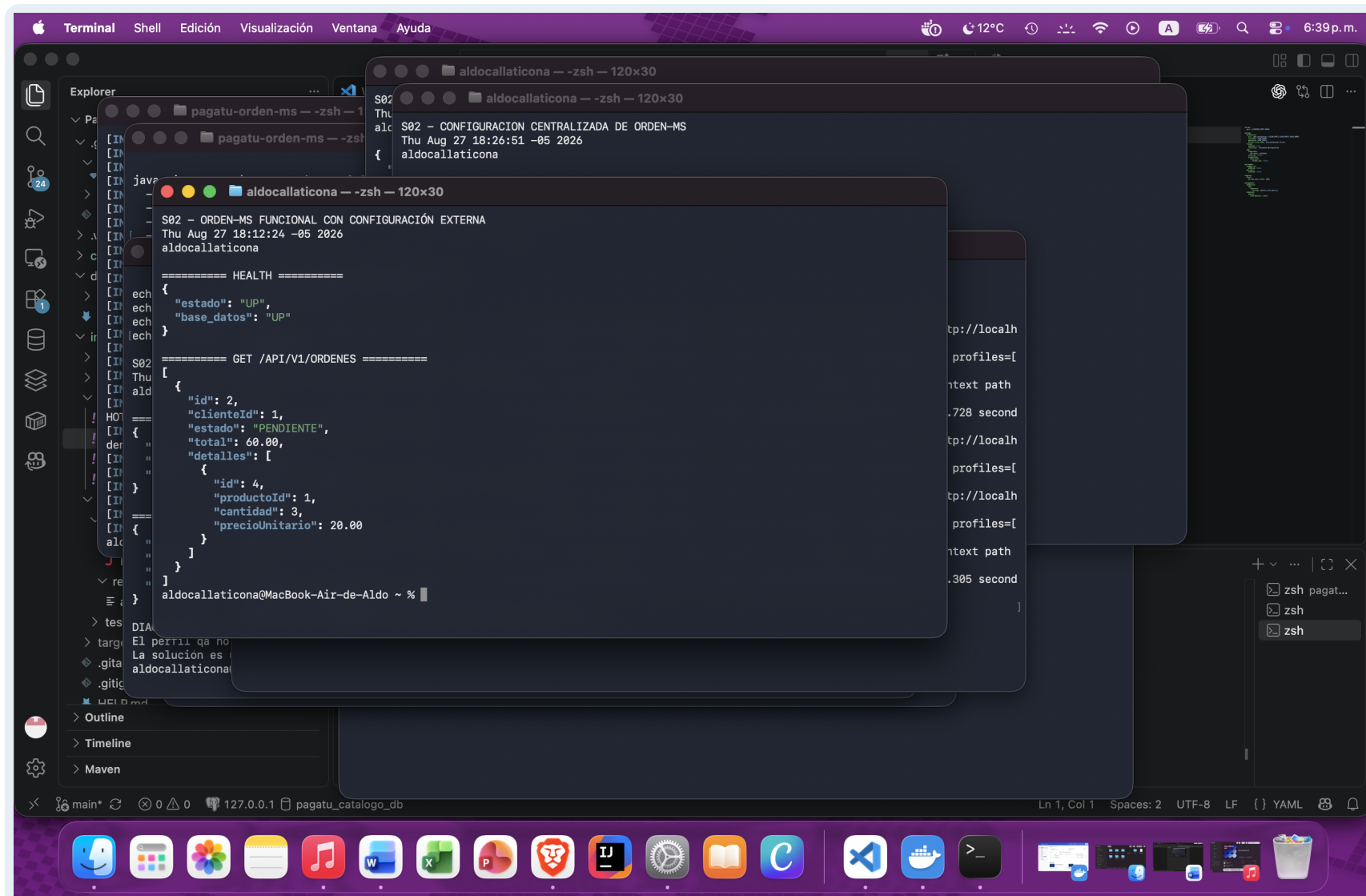
Evidencia: consulta de /orden-ms/dev y /orden-ms/prod sobre el repositorio de configuracion. **Hallazgo:** DEV usa valores locales; PROD delega puerto y base de datos a variables de entorno.

orden-ms conectado como Config Client



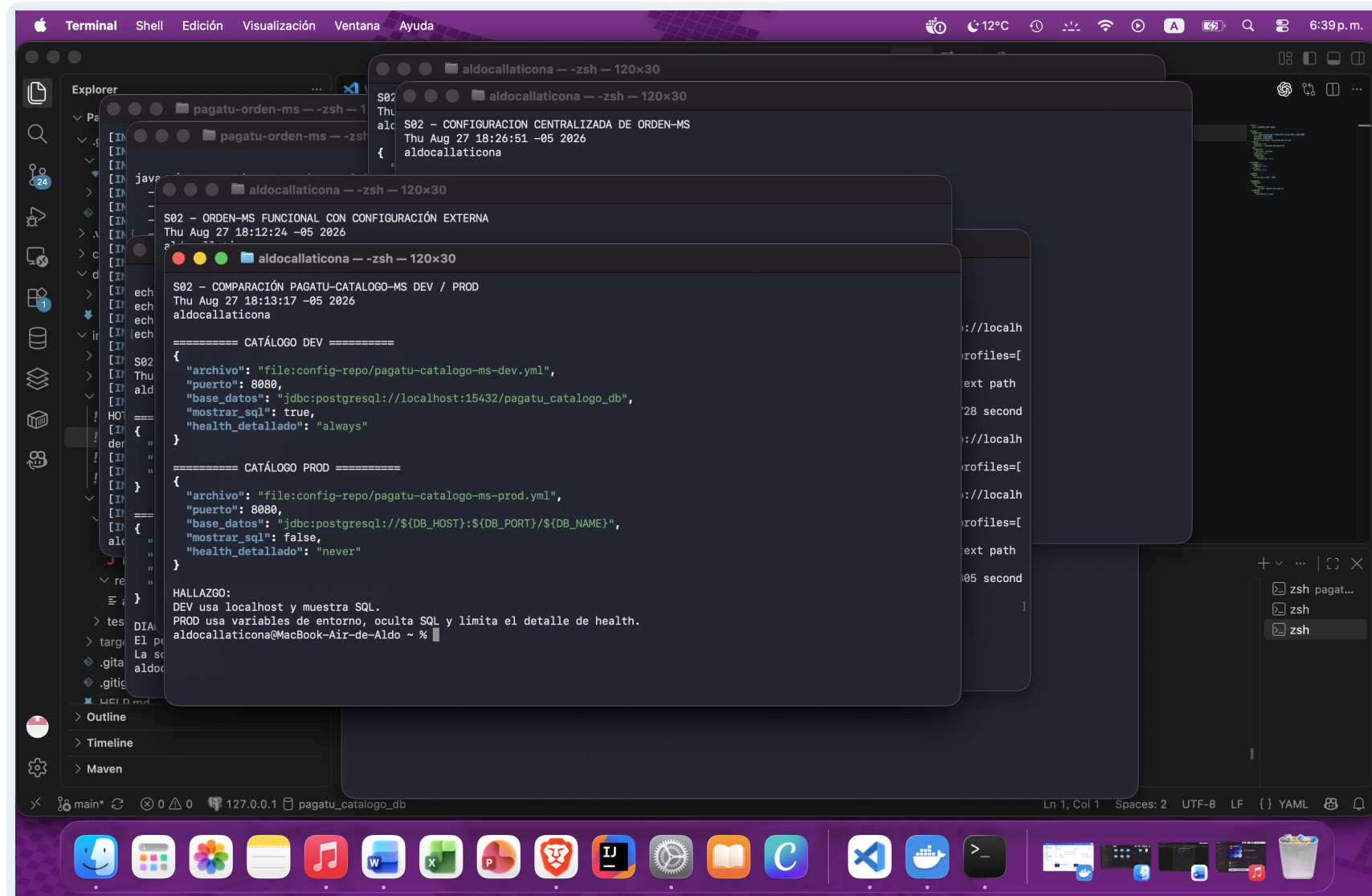
Evidencia: logs de arranque que muestran la consulta a <http://localhost:18888> y la localización del perfil dev. **Hallazgo:** el arranque vigente termina correctamente en el puerto 8080.

Servicio funcional con configuracion externa



Evidencia: health del microservicio y GET /api/v1/ordenes ejecutados contra la instancia configurada externamente. **Hallazgo:** aplicacion y PostgreSQL responden UP; el endpoint devuelve datos persistentes.

Comparacion real de catalogo DEV y PROD



Evidencia: valores servidos por los perfiles pagatu-catalogo-ms-dev y pagatu-catalogo-ms-prod. **Hallazgo:** se externalizaron URL JDBC, SQL visible y detalle de health segun el ambiente.

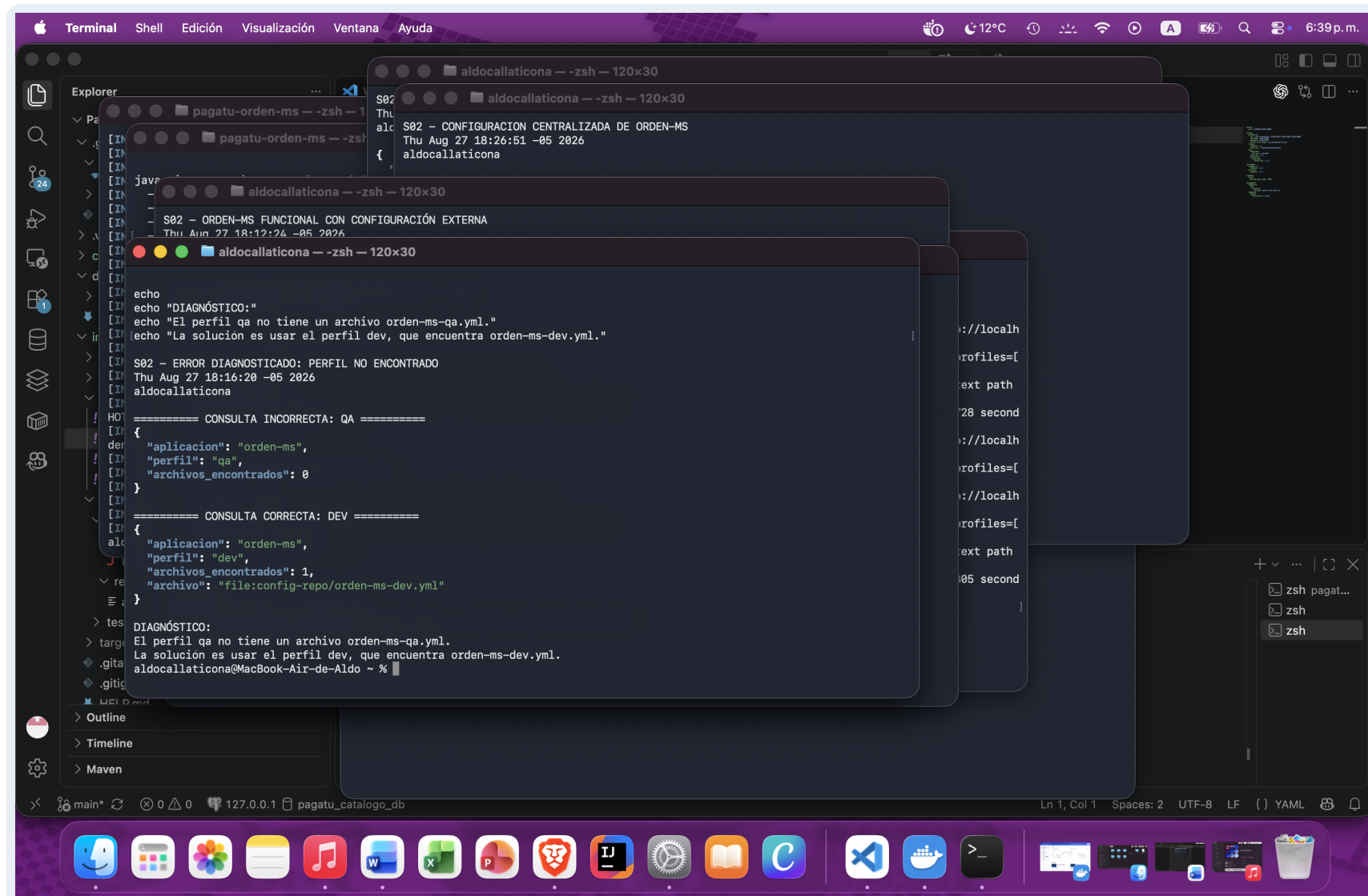
Separacion entre codigo y configuracion

Principio aplicado. El codigo define el comportamiento del servicio; la configuracion define como ese mismo artefacto se conecta y se comporta en cada ambiente. Por eso el microservicio conserva solo su identidad, perfil e importacion del Config Server, mientras los puertos, URLs JDBC, logging, Actuador y Swagger se mantienen en el repositorio central.

Aspecto	DEV	PROD	Beneficio
Base de datos	localhost y puerto fijo	DB_HOST, DB_PORT y DB_NAME	Despliegue portable
SQL	show-sql: true	show-sql: false	Diagnostico sin ruido productivo
Health	show-details: always	show-details: never	Menor exposicion de informacion
Swagger	Disponible	Deshabilitado	Superficie reducida en produccion
Artefacto Java	El mismo JAR	El mismo JAR	No requiere recompilacion

Conclusion tecnica: al promover una version no se modifican ni duplican fuentes. El ambiente selecciona su perfil y Config Server entrega valores versionados. Esto reduce configuracion divergente, permite auditoria en Git y facilita replicar instancias con el mismo contrato operativo.

Error diagnosticado: perfil inexistente



Evidencia: comparacion entre la consulta incorrecta qa y la consulta correcta dev. **Hallazgo:** qa retorna cero fuentes porque no existe orden-ms-qa.yml; se debe usar dev o crear el archivo con el nombre exacto.

Reflexion y conclusiones

Reflexion personal

1. Config Server establece un punto unico y versionable para administrar propiedades compartidas.
2. Cuando aumentan los microservicios e instancias, evita repetir manualmente la misma configuracion.
3. Los perfiles DEV y PROD permiten cambiar infraestructura sin alterar ni recompilar el codigo.
4. Cada instancia puede iniciar con el mismo artefacto y recibir valores coherentes para su ambiente.
5. La trazabilidad en Git ayuda a revisar cambios, identificar responsables y recuperar versiones anteriores.
6. Las pruebas de health, endpoints y errores reducen el riesgo de desplegar configuraciones incompletas.
7. En conjunto, la solucion disminuye el configuration drift y simplifica el escalamiento horizontal.

Entregables verificados

-  Config Server UP
-  orden-ms DEV/PROD
-  Config Client activo
-  Health y CRUD
-  Catalogo DEV/PROD
-  Error documentado

Repositorio:

<https://github.com/Aldo-Ct/Pagalotu>

Referencia de la actividad:

262dist.github.io/pagatu/sesiones/S02_Configuracion_Centralizada_Ambientes/